F | 交付、观察与恢复

你把已经能运行的工单程序交给另一位同事。他启动了进程，却打不开页面；改好端口以后，重新建立运行实例，昨天的工单又不见了。源码没有丢，问题出在源码之外：程序怎样取得运行环境、请求怎样进入、数据实际保存在哪，以及操作者凭什么判断已经恢复。

本册从这些具体问题出发。学完以后，你应能沿一个操作指出：它改变的是程序模板、运行实例、配置还是持久数据；成功证据在哪里；下一位操作者怎样重复同一条路线。

第一次只看这里

直接进入 F01。第一次只追一个运行对象：源码怎样成为可启动对象，什么证据才说明它真的可用。

本册的详细关系图、章节路线和查用说明放在全书后部。遇到‘这一步和前后哪里相连’时再回看，不需要在第一轮先读完。

目录

第一次只看这里 1

F01 | 一份程序怎样成为一次运行 3

F02 | Dockerfile 怎样把文件变成运行模板 8

F03 | 请求怎样进入，数据怎样留下 12

F04 | 进程活着，为什么还不能接单 18

F05 | 怎样证明备份能恢复，并回到已知版本 23

F06 | 怎样让另一位操作者完成验收与交接 29

把六章放回同一套运行与恢复关系 34

本册总览（需要时再回看） 37

本册知识结构总览 38

F01 | 一份程序怎样成为一次运行

本章先判断：改变或删除哪一个运行对象，会影响什么？先区分镜像、容器、进程、源码和数据，再选择观察命令。

一个程序文件可以启动两次。假设两次都执行 `python app.py`：它们读取同一份代码，却有不同的进程 ID、启动时间与运行状态。停止其中一个，不会把另一个进程或源码文件一起删除。

如果 Docker Desktop 和 Engine 可用，可以观察真实对象；如果当前没有运行时，对象图和保存的示意输出只能用于理解关系，不能写成已经运行验证。两条路径都能继续学习，但证据等级不同。

先把四个容易混在一起的对象分开：源码是“要运行的内容”，image 是“运行内容的模板”，container 是“这一次建立的实例”，进程是“实例里面此刻执行的工作”。看到源码文件，只能说明内容存在；看到 image，只能说明模板存在；只有看到正在运行的 container 和其中的服务响应，才逐步接近“这次程序可用”的判断。

先明确本轮证据路线：

- **A | 只理解关系**：先沿“镜像 → 容器 → 主进程”图追一遍，不执行 Docker。
- **B | 观察真实对象**：用已有 Engine 看一个无业务数据的入门容器，逐步取得运行证据。

请先选 A 或 B。若选 B，第一步只运行 `docker version`，确认 Client 和 Server 都能返回；看到结果后停，不要马上创建两个容器。若选 A，后面的示意输出只作为概念证据，等以后有隔离运行时再补真实证据。

两条路线的目标相同，证据等级不同。第一次只需要回答：我现在观察的是哪个对象，改变它会不会影响源码、另一个实例或持久数据？

Docker 在这条熟悉的关系上增加了运行模板与隔离实例。本章后面会用一个安全的入门实例把这条关系落地；第一次先只把“模板”和“这次实例”分开，不预先执行完整的双实例配方。

image 提供模板，container 保存这一次实例

image（镜像）保存运行文件、运行时、库，以及默认命令等配置。例如本册的 D1 镜像包含 Python、工单源码，默认命令是 `python app.py`。它本身没有正在等待 HTTP 请求的进程。

container（容器）是从镜像建立的一次运行实例。Docker 给它分配自己的身份，为进程准备隔离的运行环境，再按选定命令启动。实例还可以获得这一次专用的环境变量、端口和数据挂载。

实例一

同一镜像 I1
↓
容器 C1
↓
主进程 P1
↓
自己的状态与临时可写层

实例二

同一镜像 I1
↓
容器 C2
↓
主进程 P2
↓
自己的状态与临时可写层

“隔离”不是自动复制一整台 Windows 电脑。这里运行的是 Linux 容器；Windows 提供你的终端和文件目录，Docker Desktop 提供相关后台环境，容器内进程按照 Linux 路径和运行约定工作。它仍然使用主机资源，也仍可能因程序或配置错误失败。

Docker 操作需要已启动的 Docker Desktop、可连接的 Engine 和 Linux containers；安装前置按书末官方入口核对。先在 PowerShell 检查：

```powershell
docker version
```

只有 Client 与 Server 两部分都正常返回，才继续下面的实例实验。找不到 `docker` 或 Server 连接失败时，先解决工具与后台连接问题；尚未准备好运行环境时，可以先沿正文追踪对象关系。

为了单独观察实例关系，可以先使用 Docker 的入门镜像。取得该镜像需要 registry 网络访问；本地已有镜像时，下面操作使用现有模板。选 B 后，先做一次只读的名字检查，确认实验名没有被已有对象占用；不要直接覆盖一个你不认识的容器。

```powershell
docker ps -a --filter 'name=^fde-f01-a$'
```

这一步只回答“这个实验名是否已经有对象”，不创建或修改容器。确认没有冲突后，再只创建一个实例：

```powershell
docker run -d --name fde-f01-a docker/welcome-to-docker
```

`run` 建立并启动实例；`-d` 让它在后台运行，终端返回后进程仍可继续；`--name` 给这次实例一个便于查找的名字。看到容器 ID 后先停，确认这次确实出现了一个新对象，再用一次 `inspect` 查看它的镜像和状态：

```powershell
docker inspect --format '{{.Id}} {{.Image}} {{.State.Status}}' fde-f01-a
```

这条输出中，`.Id` 是本次实例的身份，`.Image` 是它实际使用的镜像身份，最后一项是当前状态。这里没有发布网页端口，因为当前观察的是对象身份；F03 才沿请求解释端口。

如果想亲自核对“一个模板能产生两个独立实例”，再做下面的可选对照，不必与第一次观察同时进行。先查询 `fde-f01-b`，确认没有同名对象；若已有对象，不删除它，换一组未占用的实验名称后再做。

```powershell
docker ps -a --filter 'name=^fde-f01-b$'
```

确认名称空闲、前面的 `fde-f01-a` 仍存在且属于自己的实验后，让新实例明确使用前一个实例的同一镜像：

```powershell
$f01Image = docker inspect --format '{{.Image}}' fde-f01-a
if ($LASTEXITCODE -ne 0) { throw '没有取得第一个实例的镜像身份。' }
docker run -d --name fde-f01-b $f01Image
docker inspect --format '{{.Id}} {{.Image}} {{.State.Status}}' fde-f01-a
  fde-f01-b
```

`$f01Image` 暂存刚查到的镜像 ID，不是另一份镜像。若 `run` 失败，停在错误处，不把后面的查询当作创建成功。成功时，两行 `.Id` 不同，`.Image` 相同：实例不同，模板相同。它们并不共享同一进程、日志或临时文件。

终端里的命令由谁执行

PowerShell 先找到 `docker` 这个命令行程序。Docker CLI 是 client，它向 Docker daemon 发请求；daemon 是后台管理服务，负责镜像、容器、网络与存储对象。

> PowerShell 的命令
> → Docker CLI
> → 当前连接的 Docker daemon
> → 创建 C1，选择 I1，启动 P1

现在回看前面的 `docker version` 检查，就能解释它为什么要同时返回两部分。

Client 部分说明终端调用到了哪个 CLI；Server 部分说明它连接的 daemon 版本。只有 Client、随后出现连接错误时，命令行程序存在，但后台服务没有被成功联系。此时问题还没走到应用源码、端口或数据库，不应因而判断镜像损坏。Windows 找不到 `docker`，则更早停在命令解析处。

Docker Desktop 界面能帮助浏览这些对象，但界面中的一个绿色图标不能说明工单已保存。工具入口、实例运行、业务结果需要各自的证据。

主进程退出时，容器为什么停了

现在设想镜像命令被写成 `python --version`。这条命令能正常执行：打印 Python 版本，然后返回退出码 0。Docker 成功创建了容器，也成功启动了进程；进程结束后，容器就进入 exited 状态。

> 创建成功 → 打印版本 → exit code 0 → 容器停止

退出码 0 表示进程按自身约定正常结束，不表示一个 API 会一直提供服务。要持续处理请求，入口必须是会保持服务的程序，例如后面配套 `app.py` 的 `serve_forever()`。

这时不同命令各自回答一个问题：

命令	观察的问题
`docker ps`	当前哪些容器正在运行
`docker ps -a`	已停止的实例是否仍存在
`docker inspect 名称`	它用了什么镜像、命令，处于什么状态，退出码是什么
`docker logs 名称`	程序写到标准输出与标准错误的内容是什么

若 `ps -a` 显示 Exited (0)，日志只有版本，inspect 命令也是 `python --version`，证据共同指向“选错了长期服务入口”。反复重启这个实例，只会再次打印版本并退出。

停止与删除分别留下什么

现在只停止并删除自己的第一个实验实例。再次核对 `fde-f01-a` 的身份后执行：

```powershell
docker stop fde-f01-a
docker rm fde-f01-a
```

`stop` 使 C1 的主进程停止，实例信息仍可查看；`rm` 删除 C1 本身及其可写层。镜像 I1 没有被删除，主机源码更不属于 C1 的可写层。如果做过可选对照，C2 也没有被这两条命令指向；观察完后，单独核对 `fde-f01-b`，再用同样的 `stop`、`rm` 依次清理它。没有创建第二个实例，就没有这一步清理。

如果 C1 只在自己的临时目录写了一张便条，删除 C1 就不应再期待那张便条存在。需要保存工单时，应把数据放到生命周期独立的存储；F03 会实际追踪这条路径。镜像可以再造运行实例，却不会自动包含上一个实例后来写入的所有业务数据。

到这里，最值得能预测的是：改变或删除一个对象，为什么不一定改变其余对象。`run`、`ps`、`inspect`、`logs` 会在后续操作中反复使用；长输出只查当前判断需要的字段。

> **第一次学到这里先停。** F01 的主线已经完成：镜像是可重复使用的模板，容器是这次实例，主进程决定它是否继续运行；停止或删除一个容器，不等于删除镜像、源码或另一个容器。先能根据“我正在观察哪个对象”选择 `ps`、`inspect` 或 `logs`，再进入 Dockerfile、端口、卷和 Compose。没有 Docker Engine 时，停在关系图也算完成了本章的概念路线，但不能把它写成真实运行证据。

F02 | Dockerfile 怎样把文件变成运行模板

同事能拿到 `app.py`，并不表示他已经有正确的 Python 和依赖。镜像把运行文件与默认约定放到一个可识别的产物中；Dockerfile 说明如何从明确输入建立这份产物。

这一章先追踪一次 build：它读取什么，在哪里安装依赖，最后把哪个命令交给容器。然后再解释怎样复用构建结果、限制不必要权限，以及为什么镜像身份需要记录。

builder 能看见哪些文件

从这里起使用本册独立的 `support-stack`：工单 API 接收并保存合成工单，另一个本地服务提供容量查询。它不是 E 册的 AI 助手，也不向外部创建预约；本册用它观察构建、运行与数据恢复。

先从资料文件夹中可见的 `02_配套代码_F与综合应用` `/examples/f_delivery_observability/support-stack` 整个复制到自选的独立实验目录。`02_配套代码_F与综合应用` 是底层 `02_配套练习_FX新版` 的入口，二者指向同一份配套；后文文件内若出现底层名称，不需要再复制另一份。后续命令都在实验副本的根目录执行，参考资料包保留原样。配套只需 Python 3.13 的标准库，`requirements.txt` 记录这一依赖边界；本章由 Dockerfile 准备镜像内的 Python，后续直接在 Windows 运行配套脚本时才使用主机 Python。

进入这个 `support-stack` 实验副本根目录后执行：

```powershell
docker build -t fde-support-stack:1.1.0 .
```

最后的 `.` 指定 build context，即本次构建可读取的本地文件范围。`-t` 为产物指定 tag，这里是 `fde-support-stack:1.1.0`。`Dockerfile` 默认从这个 context 中寻找；换用另一个 Dockerfile，并不等于自动改了最后这个输入范围。

实验目录	
	app.py、storage.py 应用源码
	requirements.txt 依赖声明
	Dockerfile 构建说明
	.dockerignore 排除规则
	tests/ 测试材料
	backups/ 运行后产生的备份

builder 不应把虚拟环境、备份和秘密文件都当成程序。配套 `.dockerignore` 排除 `.venv`、缓存、`.env`、数据库文件、backups 和 tests。这些匹配文件从构建上下文中排除，避免成为 COPY 的普通来源。它也减少传输和扫描的无关输入。

`.dockerignore` 的作用发生在构建输入阶段。若秘密已经由 `COPY . /app` 进入某一层，后面再删除文件，早期层仍可能包含它。真正的 build secret 应使用专门的 secret mount；不能用普通 `ARG`、`ENV` 或先复制后删除代替秘密管理。

一份实际 Dockerfile 怎样读

下面是配套构建文件的主要执行部分。它包含本例运行所需的权限建立，不能把 `USER` 单独复制出去而省略它的目录条件。

DOCKERFILE

```
FROM python:3.13-slim-bookworm@sha256:
  c45a22ea000adfd9cda29364bbe7edd23001ce5cc2ad15857cfbf7766943b9ca

ARG APP_VERSION=1.1.0
ARG EXPECTED_SCHEMA_VERSION=1
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    APP_VERSION=${APP_VERSION} \
    EXPECTED_SCHEMA_VERSION=${EXPECTED_SCHEMA_VERSION}

RUN groupadd --gid 10001 appgroup \
    && useradd --uid 10001 --gid appgroup --no-create-home appuser \
    && mkdir -p /app /data /backups \
    && chown -R appuser:appgroup /app /data /backups

WORKDIR /app
COPY requirements.txt ./
RUN python -m pip install --no-cache-dir --requirement requirements.txt
COPY app.py storage.py observability.py healthcheck.py data_admin.py uat.py ./
USER 10001:10001
EXPOSE 8000 9000
CMD ["python", "app.py"]
```

`FROM` 选取已有基础镜像，其中有 Python 和基础文件系统；它没有包含本项目源码。`ARG` 是构建参数，`ENV` 在这里把版本和默认设置留进 image 配置。运行时可以覆盖适合按环境改变的默认值，下一章会实际演示。

第一条 `RUN` 在镜像构建环境里创建用户与目录。10001 是本例用户和组的数字身份。`/data` 稍后要保存数据库，所以默认用户需要有写入该位置的权限。容器挂入的实际存储也要满足访问权限；Dockerfile 中目录可写，不代表任意 bind mount 都可写。

`WORKDIR /app` 让后面的相对目标和默认进程工作目录落在 `/app`。它没有把 Windows 的 PowerShell 切换到某个目录。第一条 COPY 从 context 复制依赖清单，安装命令随后在镜像环境读取它。配套只用标准库，清单因此没有第三方包；这种结构仍清楚分开依赖输入与应用源码。

第二条 COPY 加入运行所需模块。最后 `USER` 设定默认进程身份，`CMD` 设定默认入口。`CMD` 的方括号是 exec form：可执行程序和参数分别成为数组元素，而不是要求 shell 再解释一整串文字。

RUN 在 build 时执行；CMD 在启动容器时提供默认命令。 把 API 长期服务写进 RUN，builder 会等待该命令结束；把依赖安装当作唯一 CMD，则每次启动只做安装，安装结束后实例也结束。

`EXPOSE 8000 9000` 记录预计使用的端口元数据。它没有把 Windows 的 18080 端口连接过来，也没有启动一个监听进程。真正监听由应用完成，主机发布由运行配置完成。

完整变化可以沿文件追踪：

依赖文件

实验目录的 requirements.txt
↓
image 的 /app/requirements.txt
↓
build 时 Python 安装依赖

应用文件

实验目录的 app.py 等模块
↓
image 的 /app/*.py
↓
run 时在 /app 执行 python app.py

为什么先复制依赖，后复制源码

假设只把一个错误回复的文字改了，依赖清单不变。builder 能复用前面相同输入的构建结果，从源码 COPY 附近重新处理；不必因为 source 改变就重新安装相同依赖。

反过来，如果 requirements.txt 改了，安装步骤的输入改变，后续依赖它的结果需要重新计算。这个机制叫 build cache：它复用可复用的中间结果，不是判断业务代码正确。

image 中的文件系统变化由 layers 保存。COPY、RUN 等指令会形成文件变化；CMD、ENV 等还会设置镜像配置，不能把每条 Dockerfile 指令都理解成新增业务文件的层。构建日志里的 `CACHED` 说明某步复用了结果，不能说明依赖没有漏洞或 API 验收已通过。

`--no-cache` 要求这次不复用已有构建缓存；`--pull` 尝试取得指定 base 引用。它们并不等价。尤其本例基础镜像固定到 digest，`--pull` 仍然取得这份指定内容，不会替你选择新的安全修复版本。

固定身份与更新并不冲突

tag 是可读名字，例如 `python:3.13-slim-bookworm`；维护者可以让它指向新内容。`@sha256:...` 是内容身份，固定它可避免标签移动悄悄改变 base。代价是更新需要明确进行：核对新引用、修改、重建并验证。

base 的 digest 只识别基础镜像。最终工单 image 还包含源码和配置，要另取自己的 ID：

```powershell
docker image inspect fde-support-stack:1.1.0 --format '{{.Id}} {{json
  .Config.Cmd}} {{json .Config.User}}'
```

应看到一个 `sha256:` 身份、命令数组 `python app.py`、用户 `10001:10001`。把实际 ID 记录为 D1。D1 是本文方便解释的代号，不是可以原样传给 Docker 的镜像名字。

这些检查有明确用途：Cmd 防止默认入口选错；User 防止以不必要的 root 身份运行；ID 让以后能核对运行的确实是这份产物。任何一项都不能独自证明应用正确或全面安全。

改了文件以后应该做什么

实际改变	下一动作为什么这样选
source、依赖或 Dockerfile 改变	rebuild 产生新镜像，再用它建立新容器
本次端口或环境变量改变	recreate 应用新的运行配置；源码不变时无需另建镜像
同一实例需要重新启动进程	restart；它仍保留原镜像和原运行配置

build 成功没有把所有已存在实例自动换成新镜像。这个界限稍后也决定回滚怎么做：先查错在模板、运行配置还是数据，再选择改变相应对象的动作。

F03 | 请求怎样进入，数据怎样留下

现在已有 D1，它包含同一份 app.py。我们要用它运行两项服务：API 监听 8000，容量服务监听 9000。Windows 客户端从 18080 进入 API；API 再按服务名称访问容量；数据库写到可在实例替换后继续使用的位置。

这三段可以共用一份运行声明，但每一段解决的问题不同。先把一个请求送到 API，再接上内部服务，最后追踪工单文件。

环境变量怎样变成进程行为

配置不是只写给人看的备注。配套 app.py 在启动入口读取环境变量，选择服务角色、监听地址和端口。下面恢复实际入口中的相关语句；`ReusableThreadingHTTPServer` 是配套定义的 HTTP server 类，`ServiceHandler` 负责处理路由。

PYTHON

```
role = os.getenv("SERVICE_ROLE", "api")
listen_host = os.getenv("LISTEN_HOST", "0.0.0.0")
default_port = 9000 if role == "dependency" else 8000
listen_port = environment_int("LISTEN_PORT", default_port)
server = ReusableThreadingHTTPServer(
    (listen_host, listen_port), ServiceHandler
)
server.serve_forever()
```

`os.getenv` 从当前进程环境取文字，没有该变量时使用后面的默认值。端口要交给网络接口，因此 `environment_int` 将文字转成整数；不能转换就停止启动。`SERVICE_ROLE=dependency` 时 handler 提供容量接口，`SERVICE_ROLE=api` 时 handler 提供工单接口。

同一 image 因而可以启动不同角色，不需要把 API 版和容量版源码各复制一遍。环境变量改变的是这一次进程取得的设置。原进程已经读取过旧环境；修改 Compose 文件后，需要用新声明建立实例，单纯 restart 不会重新注入新的变量。

Compose 把相关实例放进一份声明

Compose 用 YAML 描述一组服务。缩进表示从属关系，`key: value` 表示字段，`-` 表示列表项；它是数据声明，不是从第一行逐条执行的脚本。下面是配套的核心运行关系，build 和健康细节分别在前后章节展开。

```yaml
name: fde-fx-ticket-lab
services:
  fake-dependency:
    image: fde-support-stack:1.1.0
    environment:
      SERVICE_ROLE: dependency
      LISTEN_HOST: 0.0.0.0
      LISTEN_PORT: "9000"
      DEPENDENCY_MODE: ok
  api:
    image: fde-support-stack:1.1.0
    environment:
      SERVICE_ROLE: api
      LISTEN_HOST: 0.0.0.0
      LISTEN_PORT: "8000"
      DATABASE_PATH: /data/tickets.db
      DEPENDENCY_URL: http://fake-dependency:9000
      EXPECTED_SCHEMA_VERSION: "1"
    ports:
      - "127.0.0.1:18080:8000"
    volumes:
      - ticket-data:/data
volumes:
  ticket-data:
```

`api`、`fake-dependency` 是 service 名称；Compose 按各项定义创建容器。`image` 选择模板，`environment` 传入运行差异。两项服务选择同一 image，却取得不同角色和监听端口。

Dockerfile 的 ENV 也可以提供默认环境，Compose 的 `env_file` 或运行命令也有注入入口。因此应区分两件事：主机 `.env` 可以供 Compose 替换 `${变量}`；应用进程实际拿到什么，则由镜像默认值与本次注入、覆盖规则共同决定。仅把一个 `.env` 放进目录，不会自动把其中所有内容交给程序。

18080 与 8000 分别属于谁

端口声明 `127.0.0.1:18080:8000` 分成三部分：只在主机 loopback 发布、主机端口 18080、容器目标端口 8000。客户端仍请求 Windows 上的地址，Docker 转发后才到容器进程。

```
Windows 客户端
GET http://127.0.0.1:18080/ready
        ↓ 主机发布规则
API 容器网络接口 :8000
        ↓ 应用监听 0.0.0.0:8000
ServiceHandler 处理 /ready
        ↓
HTTP response 沿连接返回客户端
```

容器里的 `0.0.0.0` 表示在该网络空间的各接口监听。若应用改为监听容器内的 `127.0.0.1:8000`，它只接收来自自身 loopback 的连接；从主机映射到容器网络接口的请求可能到不了这个 listener。此时容器仍可 running，发布规则也存在，问题在监听地址。

这种失败要沿请求路径查：客户端 URL 是否正确，主机发布端口是否存在，容器端口是否与应用一致，进程是否监听可达接口。删除数据库不可能修复这段网络关系。

Dockerfile 的 EXPOSE 只说明预计端口。你可以把主机端口改成 18082 而继续转到容器 8000；客户端随之改访问 18082，应用内部不用改变。端口左边与右边没有必须相同的理由。

API 为什么用服务名称，而不用 localhost

API 收到工单以后读取 `DEPENDENCY_URL`，交给 `dependency_capacity`。该函数在基础 URL 后面加 `/capacity`，发出有超时限制的 GET。模拟服务返回 `{"status":"available","slots":3}`；本例仅检查解析结果是 JSON 对象且 `status == "available"`，`slots` 不参与决定。因此它证明的是这个缩小接口约定下的可用状态，不代表已经验证完整容量字段合同。

```
api 容器
→ http://fake-dependency:9000/capacity
→ 同一 Compose 网络的 DNS 解析 service 名称
→ 当前 fake-dependency 容器的 9000 端口
```

这里访问的是内部容器端口，不绕回 Windows 的 18080。容量服务没有向主机发布端口，API 仍可通过共同网络访问它。

API 容器里的 localhost 指 API 自己，容量容器里的 localhost 指容量服务自己，Windows 的 localhost 又指 Windows。三个位置可以写成相同文字，却不是同一条网络接口。把依赖 URL 写成 `http://localhost:9000`，API 就会敲自己的门。

容器重建以后 IP 可以变化。服务名称让调用方按稳定名称重新解析当前实例，所以不应把一次 inspect 看到的 IP 写死到源码。已经建立的旧连接不会自动变成新连接；真实调用仍要处理连接失败并重新连接。

配套还用 `depends_on` 的 `condition: service_healthy` 让 API 初次启动前等待容量容器的指定健康检查。它的作用是稳定启动起点，不是保证容量以后永不失效。容量的 `/live` 健康只检查进程响应，即使 `/capacity` 返回受控 503，它仍可 healthy。这是下一章区分“进程存活”和“业务能力”的现成例子。

SQLite 写入的文件实际属于谁

配套创建工单时调用 `create_ticket`，该函数通过 `DATABASE_PATH` 打开 SQLite，再 INSERT 设备、症状和状态。正常提交后，数据库分配 T-42，HTTP response 才返回创建结果。

`DATABASE_PATH=/data/tickets.db` 说明应用要打开容器中的路径。`ticket-data:/data` 则把 Docker 管理的 named volume 挂到 `/data`。挂载完成以后，应用在这个目录内写文件，写到的就是挂载存储。

> INSERT 一张工单
> → SQLite 文件 /data/tickets.db
> → /data 是挂载点
> → named volume ticket-data
> → 新 API 容器重新挂载同一 volume
> → 仍能 GET 原 T-42

named volume 有自己的生命周期。删除 API 容器不等于删除它。Compose 的 `ticket-data` 是逻辑名，实际名称通常带 project 前缀；辨认时要看 inspect 与项目标签，不能猜 Docker Desktop 在磁盘上的内部目录。

反例是把数据库写成 `/app/tickets.db`，同时没有在 `/app` 挂存储。这样的文件属于某次容器的 writable layer。停止再启动同一容器时它通常仍在；删除再建立实例后，新实例从镜像内容开始，旧实例的工单不会自动出现。

bind mount 解决另一种需要：让容器访问一个操作者明确选择的主机目录。配套管理员把实验副本的 `backups/` 挂成 `/backups`，便于在 Windows 保存和检查备份。Windows 路径与 `/backups` 是挂载两端，不是同一种路径写法；含空格的主机路径应作为一个完整参数传递。

当前要保存什么	本例怎样选择
可丢弃的临时文件	容器自己的可写层
应用持续读写的工单数据库	named volume，由 Docker 管理存储
操作者需要取得的备份文件	明确主机目录的 bind mount

持久化覆盖的是实例替换后的保留。应用误删数据、操作者删除 volume 或主机存储损坏时，还要依靠独立备份与恢复流程。

怎样用一次运行把声明与结果接起来

先在全新实验 project 中检查声明，再启动。配套完整 compose.yaml 包含本章省略的 build 和 healthcheck 配置；不要拿前面的关系切片替换完整文件。

主机侧的 `healthcheck.py` 和 `uat.py` 不会自动使用 Dockerfile 里的 Python。先在实验副本根目录运行下面第一行，确认实际启动的是自己选定的 CPython 3.13；路径和版本分别是两项证据。若实验副本已有符合条件的 `.venv`，所有 `python` 调用都改用 `& .\.venv\Scripts\python.exe`。若找不到解释器、打开商店或版本不符，先停在 A09 的解释器核对处；创建虚拟环境也需要一个已经可用的基础解释器，不能用它跳过这个前置条件。

```powershell
python -c "import sys; print(sys.executable); print(sys.version)"
docker compose -p fde-fx-ticket-lab config --quiet
docker compose -p fde-fx-ticket-lab up -d --wait --wait-timeout 30 --build
python healthcheck.py --url http://127.0.0.1:18080/ready --wait-seconds 30
docker compose -p fde-fx-ticket-lab ps
```

`config --quiet` 成功说明声明能解析；不证明端口空闲、镜像可启动或数据库可写。`up` 创建运行对象，`--wait --wait-timeout 30` 等待已配置的服务运行与健康条件，限制本次 Compose 等待；随后的主机 HTTP 检查再经过 Windows 发布端口。两次检查的位置不同。每条命令失败都应停在这一处查看结果，不继续执行后面的写入。

完整验收前，先用配套 baseline 命令核对数据库为空、下一编号是 T-42。随后执行 UAT 创建并读回工单；F06 给出完整顺序。若要验证数据跨实例保留，记录旧容器 ID，再 force-recreate API，等待 ready，读取原 T-42，同时检查新 ID 与 mount。新建一张工单不能代替读回旧工单，因为它会掩盖数据是否保留下来。

你现在已经能指出同一个请求中的三个位置：主机如何进入 API、API 如何查询容量、SQLite 如何写到实例之外。网络、配置和存储在同一应用里共同工作，但不能用其中一段成功代替其余两段证据。

F04 | 进程活着，为什么还不能接单

假设 API 进程能够响应 `/live`，但 `/ready` 返回 503，理由是 `schema_mismatch`。重启以后结果仍然相同。这不是互相矛盾的信号：存活请求能执行，数据库却不满足当前程序的版本要求。

先把这个失败完整解释，再比较另一种失败：容量服务不可用时，API 仍可以安全保存工单并转人工。

就绪检查怎样阻止不兼容的写入

配套数据库的 metadata 表保存 `schema_version=1`。受控失败版本 D2 期望 2，它没有执行迁移，只改变期望值。创建工单路由在读取请求正文和调用容量之前，先执行 `core_readiness()`。

> POST /tickets
> → core_readiness
> → actual version 1，expected version 2
> → 返回 ready=False、reason_code=schema_mismatch
> → 路由返回 HTTP 503
> → 没有调用容量，也没有 INSERT 新工单

`/live` 只证明进程可以处理这个探针；它不查询容量，也不把数据库版本不兼容当作进程已死亡。`/ready` 则查询核心数据条件。它们的语义由应用决定，Docker 不会自动替你设计。

下面是可在实验副本根目录保存、直接运行的完整 Python 小例子。它使用配套 storage 模块和系统临时数据库，不启动容器，也不会留下教材目录里的数据库。先看预期 1，再只把预期改成 2。

```python
from pathlib import Path
from tempfile import TemporaryDirectory

from storage import initialize_database, readiness_status

with TemporaryDirectory(prefix="fde-f-ready-") as temporary:
    database = Path(temporary) / "tickets.db"
    initialize_database(database)

    normal = readiness_status(database, 1)
    mismatch = readiness_status(database, 2)

    print(normal["ready"], normal["reason_code"])
    print(mismatch["ready"], mismatch["reason_code"])
    assert normal["ready"] is True
    assert mismatch["reason_code"] == "schema_mismatch"
```

预期输出是 `True ready` 与 `False schema_mismatch`。同一份数据，两种程序期望，产生不同接单判断。临时数据库被删除前，代码没有为了让新期望通过而偷偷改版本号。

仅检查版本号仍不充分：如果某人把 tickets.device 列改名，metadata 可能仍写着 1。新版配套还用 `PRAGMA table_info` 核对三张核心表的列名、类型、NOT NULL、默认值和主键，并确认 tickets 的自增编号约定。真实结构与约定不符时返回 `schema_structure_mismatch`。这些检查保护本例声明的结构，不是对任意数据库所有触发器、业务约束和未来版本的万能兼容证明。

结构与版本通过后，readiness 再打开一个短事务，在 readiness_probe 表写入探针值，随后 rollback。这证明当前能取得写入条件，又不留下业务记录。它不消耗工单编号。数据库锁超出短等待、路径不可访问等问题会使检查失败；某一时刻探针通过，也不能保证下一次请求期间没有新的故障。

可选容量失败为什么得到不同结果

当前最低服务承诺是“工单可以持久保存；无法确认容量时交人工”，所以可选容量失败不应阻止整个 API 就绪。处理路线是：

共同前置

核心数据检查通过 → 读取合成工单 → 请求 /capacity

容量可用分支

请求 /capacity
↓ 合同内的 available
capacity_confirmed

容量不可用分支

请求 /capacity
↓ 超时、不可达、错误状态或坏内容
manual_review

两个分支共同回到

将选定状态与工单一起提交 → HTTP 201

`manual_review` 不是容量确认的另一种拼法。它明确表示自动容量路线未完成，但工单已保留下来，需要人接手。配套只保存这一状态，没有实现真实组织中的人工工作队列和响应时限；交付真实系统时，需要把该状态接给实际负责的人与处理入口。

若服务合同变成“没有容量确认就绝不允许接单”，就绪与写入策略也要随之改变。先定义什么结果是安全的，再决定哪些依赖决定整体可用性，不能先背一个“所有依赖都应该进 readiness”的规则。

Docker health 是怎样得到的

Compose 为 API 配置以下探针：

YAML
```yaml
healthcheck:
  test: ["CMD", "python", "healthcheck.py",
         "--url", "http://127.0.0.1:8000/ready"]
  interval: 5s
  timeout: 2s
  retries: 6
  start_period: 3s
```

这条命令在 API 容器里面执行，因此 127.0.0.1:8000 正好指向自己。healthcheck.py 接受 `/ready` 的 HTTP 200 和正确 JSON 状态，成功退出 0，失败退出非 0。Docker 据此维护额外的 health 状态。

初始状态是 starting；一次成功使它变成 healthy；按配置连续失败达到 retries 才变成 unhealthy。start_period 给初始化留出时间，其中失败不直接累计到普通失败门槛；期间已有成功则说明启动已完成。health 输出与状态存在时间差，刚改坏配置后不能要求 inspect 瞬间显示 unhealthy。

这里的 healthy 反映 `/ready` 探针。若改成测 `/live`，同一个 healthy 文字就只代表存活检查。Docker health 本身不会替应用执行业务验收，也不会因为 unhealthy 自动撤销工单；本 Compose 还没有设置“健康失败自动重启”机制。

当前证据	能作出的判断
container running	主进程仍存在
`/live=200`	存活探针被正确响应
`/ready=200`	本例版本、核心结构与短写入探针通过
Docker healthy	指定健康命令最近取得成功，并满足当前状态规则
创建后读回关键字段一致	这一条业务保存和读取路线通过

这些证据没有简单的互相替代关系。例如一个 container 内探针成功，还没有经过 Windows 的发布端口；主机请求失败时仍需回到 F03 的入口路线。

日志怎样帮助找到最早偏离

配套每完成一次请求，就向 stdout 输出一行 JSON。创建请求的关联 ID 可以是 `R-F-42-CREATE`，读回请求使用另一个 ID `R-F-42-READ`。同一前缀帮助归组，每次 HTTP 请求仍有自己的身份。

JSON

```
{
  "event": "http_request_completed",
  "service": "api",
  "request_id": "R-F-42-CREATE",
  "method": "POST",
  "path": "/tickets",
  "status_code": 503,
  "reason_code": "schema_mismatch"
}
```

这条记录把入口、结果和失败原因连起来。它不表明 T-42 已创建；恰恰相反，此请求在写入前被拒绝。查数据库的原记录与本次是否新增，仍要用读取和状态证据判断。

日志刻意不写入工单正文、整个环境变量或任意 URL 路径。读取工单的路径规范成 `/tickets/{ticket_id}`，未知路由只记占位名，避免有人把私密文字放进 URL 后进入普通日志。只“不打印 body”还不足以控制所有日志内容。

```powershell
docker compose -p fde-fx-ticket-lab logs api | Select-String -SimpleMatch
  'R-F-42'
```

此命令沿当前 Docker 日志驱动读取 stdout/stderr。改用其他日志驱动时，应核对可见范围；日志缺失不能反推“操作一定没发生”。先保留响应、时间和 request ID，再判断从哪一处缺证据。

遇到运行中故障，可以沿“实例状态→探针实际测什么→公开 reason code→同一请求日志→业务读回”收集信息。已经有 schema_mismatch 时，继续核对镜像期望和数据版本比无理由重建更有针对性。下一章用这条明确失败作为回滚触发条件，并说明恢复前必须保留什么。

F05 | 怎样证明备份能恢复，并回到已知版本

先沿一份已确认备份做隔离恢复，读回 T-42，再写出新的 T-44；备份存在不等于已经可恢复。

升级以前，数据库里有两张工单：正常路线产生 T-42，依赖降级路线产生 T-43。现在要验证备份 B1。最有说服力的结果不是“文件有大小”，而是在另一个目标中恢复它，读回原 T-42 的所有字段，再成功创建下一张 T-44。

这里的“另一个目标”很重要。演练不能覆盖仍然正确的原数据；恢复后创建的 T-44 也不能消耗原数据库的编号。先把备份与隔离恢复走通，再讨论真正的版本回滚。

一个 volume 为什么不能给自己兜底

volume 让工单跨容器替换保存，但应用 DELETE 的仍是这个 volume 中的数据。错误迁移、误删 volume 或底层存储损坏，也都可能同时影响正在运行的数据库。备份需要复制到明确的另一个位置，并保存它来自哪个时点、哪个版本和哪份数据。

本例将 B1 放在实验副本 `backups/B1/`，经 bind mount 在管理员容器里显示为 `/backups/B1/`。它独立于数据库 volume，却仍在同一台机器；这能演练恢复，不代表主机损坏后的异地容灾已解决。

SQLite 运行时可能使用 WAL：已提交变化可能同时涉及数据库与相关日志状态。随手复制一个正在被写入的 tickets.db，不一定取得你以为的完整时点。配套使用 SQLite 的 online backup API，由数据库引擎把一致视图写入新的数据库文件。

> 源 SQLite 连接
> → source.backup(destination)
> → 关闭备份目标，形成独立数据库文件
> → integrity_check
> → 读取结构、版本、工单数、下一编号、T-42 字段
> → 计算文件 SHA-256
> → 写 manifest.json

backup 负责取得可用的一致副本；业务上应备份哪个时点仍由操作者决定。runbook 在升级前停止新的合成写入，再登记这一时点。其他数据库有不同备份机制，不能把 SQLite 的接口当成所有数据库的通用复制方法。

B1 清单里的每个值都要从数据取得

本版备份目录包含 `tickets.sqlite3` 和 `manifest.json`。清单的主要字段可以这样读；角括号是说明占位，实际产物必须保存真实值。

JSON

```
{
  "format_version": 2,
  "app_version": "1.1.0",
  "database_file": "tickets.sqlite3",
  "sha256": "<实际备份文件的SHA-256>",
  "image_identity": "sha256:<实际D1镜像ID>",
  "config_fingerprint": "SHA256:<实际配置文件展开结果的hash>",
  "schema_version": 1,
  "ticket_count": 2,
  "next_ticket_id": "T-44",
  "critical_ticket": {
    "ticket_id": "T-42",
    "device": "Printer-7",
    "symptom": "Synthetic offline signal",
    "assignment_status": "capacity_confirmed",
    "created_at": "<该记录实际创建时间>"
  }
}
```

hash 用来发现备份文件是否改变，不证明备份选对了业务数据；工单数能发现数量不符，不证明字段正确；T-42 的字段帮助核对关键业务对象。三者各有用途，不能只留一个“verified”文字。

`next_ticket_id` 也不是 `ticket_count + 42`。例如 T-42、T-43、T-44 曾创建过，随后删了 T-44，数据库只剩两张，但自增序列仍记录使用过 44；下一张应是 T-45。配套从 SQLite 序列与当前最大 ID 得出下一编号，写入备份清单，再在 verify 和 restore 时核对。恢复 UAT 读取这份结果，而不写死期待 T-43。

SHA-256 不是签名。如果备份和清单被一起替换，单纯 hash 匹配无法证明来源可信；所以 B1 还需要处于受控目录，清单与已登记版本、操作者和时间相连。本例只处理自己的合成备份。

先取得正在运行的精确身份

人类版本 `1.1.0`、镜像 tag 与内容身份不是一回事。tag 可以重新指向内容；容器的 `.Image` 则指出它真正运行的镜像 ID。

```powershell
$apiId = docker compose -p fde-fx-ticket-lab ps -q api
$imageId = docker inspect --format '{{.Image}}' $apiId
$configHash = .\scripts\Get-ComposeConfigFingerprint.ps1 -ProjectName
  fde-fx-ticket-lab
```

先确认 apiId 唯一且属于本实验、命令成功，imageId 等于升级前登记的 D1。配置脚本让 Compose 展开当前声明，输出 JSON，对其字节计算 SHA-256。它能发现本次配置内容变化；跨机器路径、Compose 输出版本等变化也可能改变 hash，所以还要保存实际配置来源与工具版本。

这些名字在恢复时承担不同作用：

身份	当前用途
application version	人读的版本标签，帮助沟通
local image ID	当前 Docker host 内核对精确 D1
Dockerfile base digest	固定构建取得的基础内容
repository digest	从 registry 分发时核对仓库内容；纯本地构建时可能没有
config hash	核对已展开的这份运行声明

没有 RepoDigest 就记录没有，不能拿 base digest 冒充工单应用的分发身份。需要另一台机器取得相同镜像时，才进一步学习 registry 的 push、pull、访问权限与 repository digest。

创建备份与验证是两个动作

在已经完成正常、降级路线的已知环境中，先登记 D1 与配置并停止新的合成写入，再执行：

这里先把输入和出口分开读：backup 的输入是已登记身份与当前数据库，输出是新的 B1 文件和清单；verify 再用清单、hash、完整性和关键字段检查这个产物。backup 成功不等于恢复成功，verify 通过也还没有证明新实例能读回业务结果，后面的隔离恢复必须继续完成。

```powershell
docker compose -p fde-fx-ticket-lab --profile tools run --rm admin backup
  --database /data/tickets.db --output-root /backups --name B1 --app-version
  1.1.0 --image-identity $imageId --config-fingerprint $configHash
docker compose -p fde-fx-ticket-lab --profile tools run --rm admin verify
  --backup-directory /backups/B1
```

`admin` 是配套专用于数据管理的 service。它使用同一应用 image，通过 entrypoint 选择 `python data_admin.py`，而不是启动 HTTP 服务；上面 backup、verify 等参数交给这个程序。`--profile tools` 选择管理用途，`run --rm` 为本次管理命令创建临时容器并在退出后清理该实例。数据 volume 与主机备份目录有自己的生命周期。

backup 拒绝覆盖同名 B1 目录。verify 重新计算 hash、检查 SQLite 完整性和本例实际表结构，并比较版本、数量、下一编号与关键字段。任一不符就停止，不能因数据库文件非空而进入升级。

已完成的备份以单个关闭后的数据库文件保存，验证以只读方式打开；运行中的源数据库不能用“它永远不会变化”的选项读取。配套只对固定备份使用 SQLite 的 immutable 读取约定，避免在只读挂载中创建日志辅助文件；该约定不是提高任意活跃数据库速度的技巧。

隔离恢复如何走到业务结果

配套提供完整脚本，避免操作者手动猜 volume 名称或把目标填成正在运行的数据库：

恢复脚本的输入是 B1、来源 image/config 和一个未占用的隔离端口；它的输出应同时包含新 volume 身份、恢复后的 T-42 读回和新 T-44 的降级结果。读命令时抓住“核对来源 → 写入新目标 → 业务读回 → 失败保留证据”四个出口，别把清理临时对象误读成恢复验证的一部分。

```powershell
.\scripts\Invoke-IsolatedRestoreDrill.ps1 -SourceProject fde-fx-ticket-lab
  -BackupName B1 -HostPort 18081
```

它先核对备份目录、当前来源 image ID 和配置 hash，再选择带随机后缀的新 volume 与新容器名称，并标注合成恢复标签。18081 是备用主机端口；已被使用就停止，不结束未知进程。

数据恢复先核对备份，再写到新目标的 `.partial` 文件。这个临时产物通过完整性、结构、数量、下一编号和 T-42 字段验证后，才建立最终 tickets.db。目标已存在、临时文件已存在或最终目标在操作途中被别人创建，都拒绝覆盖；异常时保留原有目标。文件系统不支持所需建立操作时也停止，不退化成无条件覆盖。

恢复应用随后以精确 D1 启动，但运行配置与来源不同：它使用新 volume、备用端口，并把容量依赖设为不可达。这样既不污染原数据库，也不依赖原容量服务，专门验证恢复后的读取、写入与人工处理路线。

备份与恢复目标

原环境：T-42、T-43，下一编号 T-44
↓
B1
↓
新 volume
D1 + 明确隔离配置

只在隔离目标验收

GET T-42：全部关键字段与清单相同
↓
POST 新工单：T-44、manual_review
↓
GET T-44：与创建返回字段相同

原数据库保持

原环境仍只有 T-42、T-43，下一编号仍是 T-44

脚本对就绪进行有界等待，读回原 T-42，再按清单下一编号运行 UAT。成功后仅按精确标签清理本次临时容器与 volume；加 `-KeepEvidenceResources` 可暂留给另一位操作者检查，之后仍须按输出身份处理。失败时保留隔离资源与 B1，原环境不变。

由此得到的是“这个备份在这个隔离配置下能读回并完成降级 UAT”。正常容量路线的验收另在原环境完成；两份证据组合使用，不能把隔离配置写成原 Compose 网络的逐项复原。

回滚程序什么时候不需要恢复数据

现在 B1 已经通过隔离恢复，才部署受控 D2。D2 期望 schema 2，但数据保持 schema 1，ready 返回 503。当前触发条件明确，数据没有迁移，所以恢复动作可以只换回程序与原配置，继续使用原 volume。

升级前：D1 + C1 + schema 1，T-42/T-43 正确
↓
隔离验证 B1
↓
D2 只改变程序期望，未迁移数据
↓
ready 失败，停止新的写入
↓
核对 D1 身份与 C1 配置
↓
用原 volume 重建 D1 实例
↓
ready 通过，原 T-42 字段不变，新 UAT 通过

runbook 在替换 API 前先核对 v1 tag 仍指向登记的 D1、基础配置仍是 C1，再用 `--no-build --pull never` 避免恢复过程中临时重建或拉取另一内容。替换以后还检查容器 `.Image`，不能只相信命令执行过。新 PowerShell 会话应从此前保存的清单恢复 D1、C1 值，不能现场重新计算后把新值称作“旧基线”。

如果真实升级已经把 schema 1 改成 schema 2，选择就不同了。先判断 D1 是否能读 schema 2；不兼容时，可能要把备份恢复到另一个兼容目标，再进行明确的数据切换，并处理备份时点以后的新增事实。不能用一次旧事务 rollback 撤销已经提交并运行很久的迁移，也不能保证仅换 image 就不会丢失新数据。

因此 rollback 是回到已知兼容组合，restore 是从备份建立数据目标。它们可以配合，但不是同义词。读到恢复失败时，应先问哪个对象不符合预期，再决定改程序、配置还是数据。

这一区分也决定了证据顺序：先证明 B1 能在隔离目标读回，再决定 D2 是否只需切换程序与配置。若 schema 已经发生不可逆迁移，旧 image 的存在和 tag 的名字都不能替代数据兼容性判断。

F06 | 怎样让另一位操作者完成验收与交接

先追 clean baseline、readiness、create/read UAT 和失败出口；长恢复命令服务于这条判断。

“运行 Compose，确认成功”留给操作者太多隐含判断：哪个目录，哪个项目，等到何时，看到什么结果，失败了还能不能继续。runbook 是把这些决定写成可执行的操作说明。它的使用者可以不知道你开发时经历了什么，但应满足明确列出的工具与权限前置。

本章把前面的机制连成一条完整合成验收。完整命令和失败出口在实验副本的 RUNBOOK.md；正文解释为什么按这个顺序、每次结果交给谁。

先写客户要看到的结果

UAT 是用户验收测试。它从客户可见任务出发，而不是从一个内部函数的返回值出发。本例目标是：一张合成工单能保存并读回；容量可用时确认容量，不可用时留下人工处理状态；实例替换或回滚后，已有事实仍然正确。

首次正常 UAT 的输入完整写作：

```JSON
{
  "device": "Printer-7",
  "symptom": "Synthetic offline signal",
  "data_classification": "synthetic"
}
```

`data_classification` 是程序要求的显式标记，非 synthetic 值被拒绝。它不能自动判断一句文字是否真的来自客户，所以操作者仍只输入自己的合成材料。本例也没有登录系统，不把未实现的身份认证当作已验收能力。

四次 HTTP 交接如下：

```
GET /live        R-F-42-LIVE
→ GET /ready     R-F-42-READY
→ POST /tickets  R-F-42-CREATE
→ GET /tickets/T-42  R-F-42-READ
```

同一前缀将它们归成一次验收，但四次请求拥有独立 request ID。创建的 response 必须返回 T-42、预期 assignment 状态、相同 device/symptom 和创建时间；读回 response 必须返回与刚创建工单完全相同的字段。请求关联 ID 也要符合当前交接。只比较编号会漏掉“编号相同、内容已经改变”的错误。

一条操作说明怎样做到可执行

例如启动步骤应该这样表达：在实验副本根目录，使用 project `fde-fx-ticket-lab` 启动；等待主机 18080 上的 `/ready`，应用等待预算为 30 秒；成功后查看服务与健康状态；失败则保留命令输出、ps 与日志，禁止继续创建工单。

```powershell
python --version
docker compose -p fde-fx-ticket-lab up -d --wait --wait-timeout 30
if ($LASTEXITCODE -ne 0) { throw 'Start failed.' }
python healthcheck.py --url http://127.0.0.1:18080/ready --wait-seconds 30
if ($LASTEXITCODE -ne 0) { throw 'Readiness wait failed.' }
docker compose -p fde-fx-ticket-lab ps
```

PowerShell 中 `$LASTEXITCODE` 保存上一条原生命令的退出码；显式判断使失败停在当前步骤，而不是继续后面的写入。探针脚本在每轮前检查剩余预算，到期不再发起请求，晚于预算返回的成功也不算预算内通过。单次 HTTP timeout 仍受底层连接、读取等 I/O 阶段语义影响；这里的预算限制后续轮询，不是能在精确墙钟时刻强杀所有阻塞的保证。Docker 自身探针另有执行超时。等待条件是可观察状态，不能仅写“睡几秒大概就好了”。

另一条重要前置是干净基线：

```powershell
docker compose -p fde-fx-ticket-lab --profile tools run --rm admin baseline
  --database /data/tickets.db
```

它只读取现有工单数与自增序列。只有 `ticket_count=0`、下一编号 T-42 才继续首次 UAT。数据库没有工单，也可能曾创建并删除过，所以只检查 count 不充分。基线失败应先辨认和保存原有合成数据，不能为让命令变绿而直接清空未知 volume。

从全新环境完成一次连续验收

本次完整路线固定为下面的状态变化。它是一个有前后依赖的实验，不是可以任意顺序重复的菜单。

先把表格当作一条输入—输出路线读：前一行留下的对象身份和数据，才是下一行的输入。正常、降级、隔离恢复和 D2 回滚各有自己的 project、volume、request ID 或基线；读到某一步失败就停在该步，保留它实际留下的证据，再决定从哪条恢复出口继续。

阶段	操作后需要留下的事实
前置检查	CLI、Server、Compose 可用；实验路径与 project 明确；端口无未知占用

阶段	操作后需要留下的事实
构建与登记	D1 的实际 image ID、Cmd、User、源码或交付包 hash、配置 C1 已记录
启动与空基线	ready 通过；数据库为空，下一编号 T-42
正常 UAT	创建并读回 T-42，状态 capacity_confirmed，四次请求可关联
替换 API 实例	新容器 ID、同一数据 volume；原 T-42 字段仍在
容量降级 UAT	在原数据库新建 T-43，状态 manual_review；API 仍 ready
恢复容量并备份	原环境两张工单；B1 count=2，next=T-44，关键记录正确
隔离恢复	新 volume 读回 T-42；隔离新建 T-44/manual_review；原环境不变
受控 D2 与回滚	D2 schema mismatch；回到精确 D1/C1后，原环境新建 T-44正常通过
交接	保存各份身份、结果和恢复证据，按精确对象停止运行

正常、降级、隔离和回滚使用不同 request ID 前缀。隔离环境与原环境各自创建 T-44，并不冲突，因为它们不是同一个数据库。没有发生新增写入时，读回请求也不会消耗下一编号。

可以运行配套验收入口：

```powershell
python uat.py --base-url http://127.0.0.1:18080 --request-id-prefix R-F-42
  --expected-ticket-id T-42
```

只有干净基线首次创建才使用这条命令。降级阶段在已有 T-42 的原环境中，改用 `--expect-assignment manual_review`、独立请求前缀和 T-43；隔离恢复脚本从 B1 清单读取其下一编号。详细命令按 RUNBOOK.md 顺序执行。

UAT 不是只读检查。它可能已经创建了工单，然后因为字段比对或后续读取失败而报告失败。此时原样重跑不会“继续上一条”：本例没有创建幂等接口，再次 POST 会产生新工单。应该先看失败 step 与已有返回，读回实际编号，确认已发生什么；需要重新做整条实验时，建立明确的新基线，而不是连续猜编号。

因此验收表中的“通过”必须同时指向客户可见结果和对象证据：创建 response、GET 读回、实际 container / volume 身份以及恢复后的再验证各自回答不同问题。命令跑完不自动补齐缺失的那一项。

交接清单为什么同时保存程序、配置和数据

假设同事说“我已经回到 1.1.0 了”，但他使用了另一份 compose.yaml。即使源码版本正确，端口、数据挂载和 schema 期望仍可能不同。交接清单必须让下一位操作者重新识别这些关系：

> 运行程序：源码/交付包 hash、app version、D1 image ID
> 构建输入：Dockerfile、base reference、工具版本
> 本次配置：Compose 文件、project、C1 hash、端口、挂载
> 当前数据：实际 schema、volume 身份、关键工单
> 恢复输入：B1 与清单、隔离恢复结果
> 验收结果：正常、降级、替换、恢复、回滚分别通过到哪里
> 交接信息：操作者、时间、失败时联系谁、需要保留哪些证据

无需把整个 environment 和所有 inspect 输出都放进共享材料。只保留能核对对象的字段，秘密与业务正文按实际权限另行处理。配置 hash 便于发现改变，但不是配置本身的替代；缺失原文件时，只拿 hash 无法恢复运行声明。

自动测试、当前健康与 UAT 回答不同问题。单元测试证明某个输入下的模块行为；本机双进程测试增加真实 HTTP 与 SQLite 持久性；容器运行再增加镜像、网络和挂载证据；另一位操作者按 runbook 完成路线，才暴露文档是否仍依赖口头补充。任何一类都不能独自代表其余几类。

失败出口与日常停止同样属于交付

runbook 的失败出口应把操作者停在可恢复位置。例如 B1 hash 不匹配，停止升级、保留 B1 与当前正确数据；回滚后 image ID 不符，停止写入并交出实际身份；ready 通过但业务读回错，保留 request ID 和字段差异，不宣布客户路线恢复。

正常暂停实验时：

```powershell
docker compose -p fde-fx-ticket-lab stop
docker compose -p fde-fx-ticket-lab ps -a
```

stop 保留实例，方便之后启动。要删除本 project 的容器和网络，先按 label 列出对象，再执行普通 down：

```powershell
docker ps -a --filter label=com.docker.compose.project=fde-fx-ticket-lab
docker volume ls --filter label=com.docker.compose.project=fde-fx-ticket-lab
docker compose -p fde-fx-ticket-lab down
```

普通 down 不删除本例 named volume，也不删除主机 backups。`down --volumes` 会扩大到删除项目声明的数据 volume；只有已经核对精确对象、确认全是可丢弃合成数据并明确决定丢弃时才采用。`prune` 面向更广的一类未使用资源，不适合拿来处理当前项目的一次失败。F01 的独立命名容器不属于这个 Compose project，需要按各自精确名字单独核对停止与删除。

最终请一位满足前置、没有开发历史的操作者从交付包开始执行。若他必须询问一个关键决定，就把缺失信息补回相应 runbook 位置，再从断点验证。这样得到的是可用的文档接口，也为后续练习暴露真实关系；不要求为了“教材完成”替操作者声称已经掌握全部内容。

把六章放回同一套运行与恢复关系

现在看一张已经具体可解释的图。构建产物、运行配置与持久数据在创建实例时相遇；观察与验收检查这个实际组合，恢复再根据失败对象选择动作。

构建输入成为运行模板

源码 + 依赖 + Dockerfile
↓ build
image D1

模板与这次配置相遇

image D1 + Compose 运行配置
↓
API 容器与容量容器

主机请求进入

主机请求 → published port → API handler

API 的容量分支

API handler
↓
service DNS
↓
容量读取

API 的数据分支

API handler
↓
SQLite
↓
挂载点
↓
named volume

观察这个实际组合

实际运行结果 → 状态与日志、业务读回

数据恢复到独立目标

named volume 中的 SQLite
↓ backup
备份 B1
↓
新目标 + 精确程序 + 隔离配置
↓
关键数据读回 + 新写入 UAT

需要构建时找 Dockerfile 和 context；需要改变地址时找运行配置；实例替换丢数据时找实际数据库路径和挂载；进程活着却不接单时找探针与公开失败原因；升级失败时先核对程序、配置和数据的兼容组合。这些选择来自对象关系，不来自记住一长串命令。

当前现象	回到哪条关系
CLI 存在却联系不到 Server	F01 的 client→daemon 入口
容器打印版本就退出	F01/F02 的主进程与 CMD
Windows 请求失败，容器内探针成功	F03 的 host port→container port→listener
recreate 后旧工单不见	F03 的 SQLite path→mount→volume
live 成功、ready 返回 schema mismatch	F04 的版本期望与核心接单检查
备份可读却恢复后字段不同	F05 的备份时点、关键记录与隔离读回
回滚标签正确但行为不对	F05 的实际 image ID、配置与数据兼容
操作者仍要猜等多久或失败后做什么	F06 的步骤证据与失败出口

什么时候再进入构建与分发扩展

项目需要先编译前端或二进制，而最终运行只需编译产物时，才需要进一步学习 multi-stage build：构建阶段保存工具，最终阶段只复制所需产物。此时应已理解 COPY 的来源、最后 stage 的文件、Cmd 和 User，学完回到 F02 验证实际 image。

另一台机器必须取得精确镜像时，再进入 registry、repository、push/pull、凭据与 repository digest。学完回到 F05，以目标机器实际取得的身份和 UAT 连接分发与运行证据。单机练习不必为展示这些名称提前接入外部仓库。

Kubernetes、复杂云网络、集中监控平台、多架构构建、完整签名与供应链平台不承担本例成功条件。真实项目出现相应需求时再增加它们；明确的文件来源、运行对象、数据、探针、备份和恢复关系仍然要保留。

下一本《一套应用怎样连接起来》会把这些交付关系接回 Python、HTTP、数据库、外部调用与受约束 AI。它使用自己的独立数据和完整应用，复用 T-42 作为便于追踪的公开编号。接起来的重点是上一段交出的值、状态和证据，是否被下一段按准确含义接收。

官方查用入口与版本边界

下列是持续更新的官方操作文档（live，复核日期 2026-09-07）。安装前置、具体 Compose 选项和基础镜像更新，应在实际操作时重新核对；书中的输入、实例、配置与数据关系不依赖界面恰好显示哪种颜色。

- Docker 概念与后台服务
- Docker Desktop Windows 安装前置
- Build context 与 .dockerignore
- Dockerfile：RUN、CMD、ENV、USER 与 HEALTHCHECK
- 固定基础镜像版本与更新
- Compose 网络与服务名称
- Compose 启动顺序与健康条件
- Compose 环境变量注入
- Compose config
- Compose restart 的配置边界
- 主机端口发布
- Docker volumes
- Docker 日志
- Python 3.13 sqlite3 与 backup
- SQLite URI 与 immutable 前提

Dockerfile 中的完整 base digest 是配套固定的构建输入，更新时应作为明确变更。本文的工单、验收、备份清单与恢复脚本是围绕这些公开机制构造的合成示例，不是 Docker 官方作业，也不代表真实客户环境已获得同样的验证。

本册总览（需要时再回看）

这里保留整册地图、章节关系和查用入口；第一次阅读不需要先读完。

本册知识结构总览

先按需要解决的问题分组。章节顺序帮助理解，图中的分组不是一套请求必须依次经过的执行步骤。

怎样把程序交给别人运行?	
交出的东西与正在运行的东西是什么关系?	源码、image、container、主进程、Docker 后台服务 F01 从哪些文件构建、依赖怎样安装、默认运行什么 F02
这一次运行怎样接上请求与数据?	运行配置、主机端口、容器端口、服务名称 F03 哪些文件随实例消失，哪些应放持久存储 F03
现在究竟能不能完成承诺?	进程、存活检查、就绪检查、业务验收、关联日志 F04
失败以后怎样回去，并让别人也能完成?	精确版本、备份、隔离恢复、兼容数据与回滚 F05 操作手册、客户验收、失败出口和交接 F06

构建解决“运行需要的文件从哪里来”，配置解决“这一次使用哪些地址和数据”，健康检查解决“当前能否满足指定条件”，恢复解决“怎样回到一个已知可用的组合”。它们相互配合：正确的镜像可能配错端口，正确的端口可能接到不兼容数据库；数据库恢复成功以后，应用仍需读写它才能形成恢复后的业务证据。

用一套小应用看见这些关系

配套包含两个 Python 服务。API 接收合成打印机工单；容量服务返回是否有可用容量。正常时，API 保存工单并标为 `capacity_confirmed`；容量不可用时仍保存工单，标为 `manual_review`，留给人工处理。这里查询容量，没有向外部创建预约，也没有实现 D 册的幂等写请求。

请求进入

Windows 上的请求 → 工单 API

容量查询

工单 API
↓ 查询容量
本地模拟容量服务

数据保存

工单 API
↓
SQLite
工单编号、设备、症状、处理状态、创建时间

这是一份独立的教学应用。干净数据库的第一张工单叫 T-42；编号与别册相同不表示它们共用数据库。F 的 `capacity_confirmed` 是这份小程序的状态，不能替代 X 册外部派单的 `partner_confirmed`。

本册既给出离线可追踪的例子，也给出实际运行的方法。阅读时需要理解文件、地址、状态怎样连接；完整参数、备份脚本和长清单可以查用。真正执行以后，再用当前环境的输出验证自己的判断。读完解释与独立完成操作，是不同的学习证据。